

NYUAD Meeting 3/4/2026

LLM-generated unit tests for QHack: approach, metrics, and results

1) Goal of the work

- Build an end-to-end pipeline that generates unit tests with LLMs and evaluates whether they are useful.
- Useful means: tests run, accept correct solutions, reject wrong solutions, and exercise official inputs in generated test logic.

2) How generation works (prompts + framework)

- For each challenge, the model generates 3 test files: syntactic, semantic, behavioral.
- Prompt roles: syntactic checks structure/signatures; semantic checks expected outputs; behavioral checks end-to-end `run()+check()`.
- Full reference solution is not provided in prompt (to reduce leakage/copying).
- After generation, framework injects official `TEST_CASES = [...]` at top of each file.
- During evaluation, candidate code is injected via `INJECTED_SOLUTION_CODE` (tests do not hardcode solution code).

3) How evaluation works

- Evaluator runs each generated test file with `pytest`.
- Two modes: human-only (selected human baseline only) and full (human + LLM probes labeled correct/wrong by oracle).
- Outputs are saved as JSON (detailed numbers) and Markdown (readable summary).

4) Metrics and what each one means

Metric	Simple meaning
ES	Do tests execute without crashing?
CS	Do tests accept solutions labeled as correct?
BDS	Do tests reject/fail solutions labeled as wrong? (full mode only)
TQS	Human-only: $ES \times CS$. Full: $ES \times CS \times BDS$.
C_in	After removing injected <code>TEST_CASES</code> block, does LLM-written test code use official inputs?
Div	Coverage breadth ($C_in + \text{number of test functions}$).
CQ	Combined score $\sqrt{TQS \times C_in}$.

How correctness and coverage are assessed

- Correctness is assessed by $CS + BDS + TQS$.
- Coverage is assessed by C_in (not naive full-file literal matching).

- Injected TEST_CASES is stripped before computing C_in, so coverage reflects what the LLM actually wrote.

5) Summary numerical results

Human-only evaluation (average over 8 challenges)

Model	ES	CS	TQS	C_in	CQ	Div
gemini3pro	1.00	0.96	0.96	0.71	0.67	0.71
claudeopus46	1.00	0.88	0.88	0.69	0.57	0.70
gpt54	1.00	0.83	0.83	0.71	0.58	0.71
deepseekv32	0.96	0.46	0.46	0.35	0.28	0.36
qwen3	0.79	0.38	0.38	0.44	0.12	0.43

Full evaluation: human + LLM probes (average over 8 challenges)

Model	ES	CS	BDS	TQS	C_in	CQ	Div
claudeopus46	1.00	0.88	0.80	0.68	0.69	0.55	0.70
gemini3pro	1.00	0.96	0.67	0.62	0.71	0.62	0.71
gpt54	1.00	0.84	0.71	0.55	0.71	0.56	0.71
deepseekv32	0.96	0.47	0.75	0.26	0.35	0.27	0.36
qwen3	0.79	0.38	0.50	0.08	0.44	0.08	0.43

6) How to interpret the numbers (what to say in meeting)

- High ES + CS in human-only: tests are stable and align with baseline human solution.
- Drop from human-only TQS to full TQS: weaker rejection of wrong solutions (lower BDS).
- High C_in with low TQS: tests mention official inputs but are not sufficiently discriminative.
- Lower C_in on syntactic tests can happen because they focus on structure/signatures, not input loops.
- Main findings: best human-only = gemini3pro; best full = claudeopus46; coffee_conundrum is challenging across models.

One-line conclusion

The pipeline is controlled and reproducible; metrics separate execution, correctness, bug detection, and coverage-style input exercise in generated test code.